

3 Rozsáhlý svět programovacího jazyka Python v současnosti

Pod označením Python si většina vývojářů představí jak konkrétní verzi interpretu Pythonu, včetně jeho základních knihoven, tak i de facto standard tohoto programovacího jazyka. V rámci jednotlivých vydání interpretu Pythonu se totiž do tohoto jazyka přidávají další nové vlastnosti tím, že ostatní implementace Pythonu (o nichž pochopitelně bude řeč v dalších podkapitolách) tyto změny postupně akceptují a taktéž je implementují (i když je nutno upozornit na to, že většinou s větším či menším zpožděním).

Původní interpret programovacího jazyka Python se z důvodu rozlišení mezi programovacím jazykem a jeho jednou (i když referenční) implementací dnes nazývá *CPython* (toto označení vychází z faktu, že tento interpret je naprogramován v jazyku C). Kromě CPythonu ovšem máme v současnosti k dispozici i více či méně kvalitní *překladače* programovacího jazyka Python, a to jak AOT (*Ahead Of Time*) překladače, tak i JIT (*Just In Time*) překladače. Navíc existují i další varianty Pythonu určené pro běh na virtuálních strojích Javy popř. na virtuálním stroji s CLR (tedy .NET). A aby toho nebylo málo, lze Python poměrně úspěšně provozovat i ve webovém prohlížeči.

Svět programovacího jazyka Python je tedy poměrně rozsáhlý a pravděpodobně se (možná vedle Scheme, LISPu i již zmíněného jazyka BASIC) jedná o jazyk s nejvíce různorodými implementacemi vůbec. A právě s některými zajímavými implementacemi Pythonu se ve stručnosti seznámíme v této kapitole.

3.1 Standardní interpret Pythonu: de facto standard jazyka

V současnosti je vyvíjeno či alespoň udržováno pouze pět verzí Pythonu (tedy, jak již víme, přesněji řečeno CPythonu). Jedná se o následující verze:

Verze	Stav
3.8	security
3.9	security
3.10	security
3.11	bug fix
3.12	bug fix

Pro ty verze Pythonu (CPythonu), které se nachází ve stavu *security*, se oficiálně vydávají pouze zdrojové kódy, do nichž jsou přidávány jen nové bezpečnostní opravy, ovšem nikoli již opravy

jiných chyb (a už vůbec ne nějaká další vylepšení a nové vlastnosti). To znamená, že překlad binárních balíčků v takovém případě provádí například autoři distribucí operačních systémů (Linux) atd. Naproti tomu novější verze nacházející se ve stavu *bug fix* jsou vydávány i v binární podobě a jsou do nich postupně zahrnovány i opravy běžných chyb.

Co to ovšem znamená v praxi? Zejména na produkčních systémech by měly být provozovány pouze udržované verze Pythonu, což dnes konkrétně znamená Python 3.8 či vyšší (a to za předpokladu, že pro použitý operační systém nebo pro jeho distribuci jsou skutečně ještě vydávány bezpečnostní opravy). To ovšem není vše. Pokud to pochopitelně okolnosti dovolí, je dnes vhodné používat verzi 3.11 či 3.12, protože pro tyto verze jsou vydávány opravy všech chyb. A navíc, jak uvidíme v dalším textu, došlo ve verzi 3.11 k poměrně významnému urychlení standardního interpretu Pythonu, což znamená, že například služby naprogramované v Pythonu budou vyžadovat méně zdrojů (CPU) popř. mohou být jednotlivé úkoly dokončeny rychleji. I to může být jeden z důvodů přechodu na novější verzi.

V praxi, konkrétně při nasazování různých verzí Pythonu i při plánování jejich případného upgrade, je taktéž dobré sledovat termíny, do kterých bude daná verze Pythonu oficiálně podporována. Pro všech pět v současnosti podporovaných verzí Pythonu jsou tyto termíny naplánovány následovně:

Verze	Podpora do
3.8	říjen 2024
3.9	říjen 2025
3.10	říjen 2026
3.11	říjen 2027
3.12	říjen 2028

Poznámka: nové verze Pythonu (CPythonu) jsou vydávány s frekvencí přibližně jedné verze do roka. To je poměrně nízká frekvence, která dává vývojářům, DevOps týmům či administrátorům dostatek času na přechod na vyšší verzi. Některé jiné programovací jazyky naproti tomu vychází s větší frekvencí. Příkladem může být programovací jazyk Go, jehož nové verze jsou vydávány dvakrát do roka (viz například <https://go.dev/doc/devel/release>).

3.2 Varianty Pythonu určené pro běh ve virtuálních strojích

Standardní CPython je dodáván ve formě nativního spustitelného souboru (popř. skriptu, který takový soubor spouští). Ovšem můžeme se setkat i s implementacemi Pythonu určenými pro běh ve virtuálních strojích. Konkrétně se jedná o JVM a CLR, tedy o virtuální stroje Javy (a celého ekosystému postaveného okolo Javy) a o .NET. Pro tyto virtuální stroje vznikl *Jython* a *Iron Python*.

3.2.1 Jython

Jython je jednou z implementací programovacího jazyka Python určená pro běh v rámci virtuálního stroje Javy (*JVM – Java Virtual Machine*). A nejenom to. Navíc totiž aplikace psané v *Jythonu* mohou velmi dobře kooperovat s třídami (*class*) a rozhraními (*interface*) vytvořenými v Javě, což je v praxi mnohdy velmi výhodné, protože s rostoucí složitostí moderních aplikací je většinou zapotřebí mít k dispozici vhodný skriptovací jazyk sloužící jako „lepidlo“ (*glue*) mezi jednotlivými stavebními bloky, z nichž se taková aplikace skládá.

Jython ovšem samozřejmě není dokonalý. Jednou z jeho velkých nevýhod je fakt, že je stále postaven na dnes již obstarožním Pythonu 2, druhou nevýhodou pak ta skutečnost, že se jedná o dosti pomalý jazyk (a to i v porovnání se standardním CPythonem). Tato pomalost se negativně projeví zejména při výpočtech a někdy i při manipulaci s rozsáhlými datovými strukturami, ovšem u aplikací, v nichž převládají I/O operace se ne vždy musí jednat o kritický nedostatek.

3.2.2 Iron Python

Zatímco výše zmíněný *Jython* je varianta Pythonu určená pro běh v rámci virtuálního stroje Javy, je *Iron Python* pro změnu implementován pro prostředí .NET popř. Mono. Samotný Iron Python je naprogramován v jazyce C#, zatímco původní Python je vytvořen, jak již víme, v jazyku C. Iron Python je v současnosti nabízen ve dvou verzích: 2.7 (kompatibilita s originálním Pythonem 2.7) a taktéž ve verzi 3.4 (v době psaní této knihy konkrétně ve verzi 3.4.1). Velkou předností Iron Pythonu je jeho výborná integrace s knihovnami pro .NET/Mono, a to včetně knihovny pro tvorbu grafického uživatelského rozhraní atd.

Poznámka: zajímavé je, že původní autor Jythonu i Iron Pythonu je jedna a tatáž osoba: Jim Hugunin.

3.3 Překladače Pythonu

U standardního interpretru Pythonu (CPython) se zdrojové kódy před spuštěním překládají do bajtkódu (*bytecode*) a tento bajtkód je následně interpretován. To je, zcela podle očekávání, poměr-

ně pomalé. Z tohoto důvodu postupně vznikly překladače (*compiler*) zajišťující překlad zdrojových kódů napsaných Pythonu do více či optimálně vygenerovaného nativního (strojového) kódu. Takový překlad je možné provést v jediném kroku nebo ve více fázích (potom typicky rozlišujeme *frontend* a *backend* překladače). Navíc se od sebe jednotlivé překladače liší podle toho, zda je překlad proveden ještě před samotným spuštěním aplikace (*AOT* – *Ahead Of Time*) nebo až ve chvíli, kdy se načítají jednotlivé balíčky nebo když se detekuje takzvaná „horká“ část kódu (*JIT* – *Just In Time*). Každý ze zmíněných způsobů, tedy interpretace bajtkódu, JIT překlad a AOT překlad, má své výhody a zápory a obecně pravděpodobně není možné bez znalosti vlastností konkrétního projektu říci, který z nich je obecně výhodnější. Vše záleží na konkrétním způsobu použití.

3.3.1 PyPy

PyPi je typickým představitelem *Just in Time* překladače. Zdrojový kód je tedy interpretován (spouštěn) a současně i analyzován. V okamžiku, kdy se nalezne taková sekvence kódu, kterou by bylo vhodné přeložit, je proveden překlad. Typicky se jedná o často volané funkce a především pak o (vnitřní) programové smyčky. Toto chování umožňuje reagovat na situaci, kdy se například nějaká funkce volá s různými typy argumentů a mění tak své chování. Taková funkce může být v případě potřeby přeložena v několika variantách. Určitou nevýhodou JIT překladače PyPy je fakt, že není stoprocentně kompatibilní s CPythonem. Některé programy lze v PyPi používat naprosto bez problémů (budou „jen“ mnohem rychlejší, což samozřejmě všichni uvítají), ale v dalších situacích budou nutné větší či menší úpravy projektu. Zajímavé je, že část PyPi je naprogramována v dále zmíněném RPythonu.

3.3.2 Numba

Numba je projekt primárně umožňující překlad kódu v době běhu aplikace, tedy v *runtime*. Jedná se tedy o *Just In Time* překladač, který má tu výhodu, že dokáže poměrně efektivně odvodit datové typy proměnných a parametrů funkcí na základě skutečného chování aplikace. To samozřejmě neznamená, že by JIT již při prvním volání nějaké funkce přesně věděl, jakým způsobem má tuto funkci přeložit. Ve skutečnosti se dozví pouze informace o jediné konkrétní větvi, kterou může přeložit. V případě, že bude ta samá funkce později volána s odlišnými typy argumentů, popř. se její chování změní jiným způsobem (Python je v tomto ohledu velmi dynamický jazyk), provede se just-in-time překlad znovu, takže zde zaplatíme za vyšší výpočetní výkon poněkud většími paměťovými nároky a pomalejším během prvních volání funkce (či celého projektu).

Z pohledu běžného vývojáře je největší předností tohoto způsobu překladu fakt, že není zapotřebí samotný zdrojový kód modifikovat (až na uvedení anotace před funkcí). Nepříjemný je přesun času překladu do času běhu projektu, což sice nevádí u aplikací, které běží delší dobu, ovšem u jednorázových skriptů může být použití JITu spíše kontraproduktivní (vše je nutné v praxi ověřit pomocí benchmarků).

Samotný překlad je prováděn na několika úrovních, přičemž Numba na nižších úrovních využívá možností nabízených překladačem *LLVM*.

Nástroj Numba ovšem poskytuje i alternativní způsob překladu. Jedná se o již několikrát zmíněný *AOT (Ahead Of Time)* překlad, který ovšem vyžaduje informovat Numbu o datových typech parametrů a proměnných, podobně jako je tomu v Cythonu nebo v dále zmíněném nástroji *mypyc*. V tomto ohledu se Numba do značné míry podobá dále zmíněnému překladači *mypyc*.

3.3.3 mypyc

Klasickým příkladem AOT (*Ahead Of Time*) překladače Pythonu je nástroj nazvaný *mypyc*. Tento nástroj je součástí instalace *Mypy*, viz též pátou kapitolu celou věnovanou typovým informacím. Nástroj *mypyc* pracuje přibližně následujícím způsobem:

1. Je provedena analýza původního Pythonovského zdrojového kódu s jeho překladem do AST (abstraktního syntaktického stromu)
2. Na základě AST (s typy) je vygenerován zdrojový kód reprezentovaný v jazyku C
3. Takto vygenerovaný kód je přeložen překladačem céčka (typicky se používá gcc, LLVM atd.)
4. Výsledkem je sdílená knihovna strukturovaná takovým způsobem, že ji lze využít jako takzvané *C-extension* jazyka Python (lze ji přímo volat ze skriptů Pythonu)

Výslednou sdílenou knihovnu (*DLL* či *so*) je posléze umožněno naimportovat do Pythonovského skriptu. Odpovídá totiž již zmíněné specifikaci *C-extension*. Na rozdíl od dalších AOT překladačů tedy není výsledkem přímo spustitelný soubor, což opět přináší některé přednosti, ale i zápory (mezi zápory patří poněkud složitější spouštění).

3.3.4 RPython

RPython získal své jméno ze sousloví „Restricted Python“, přičemž všechna omezení (*restrictions*), která RPython zavádí, jsou pečlivě vybrána z toho důvodu, aby se zjednodušila či vůbec umožnila automatická statická analýza [®]pythonovského kódu, který je překládán do nativního (objektového) kódu. Striktně řečeno tedy RPython není „skutečným“ Pythonem, protože ne všechny jazykové konstrukce standardního Pythonu jsou podporovány i v RPythonu. A omezení se kromě samotného jazyka týká i knihoven – RPython ani zdaleka neobsahuje celou standardní knihovnu Pythonu.

RPython se typicky používá primárně pro psaní interpretrů dalších programovacích jazyků. Ve skutečnosti je však možné v případě potřeby RPython použít i pro vlastní projekty, ovšem musíme se smířit s některými omezeními a problémy při snaze používat celý ekosystém postavený okolo Pythonu. Ovšem například v situaci, kdy máme naprogramován a odladěn nějaký výpočet v Pythonu, může být zajímavé ho přeložit RPythonem a využít tak toho, že výsledkem bude rela-

tivně dobře optimalizovaný nativní kód. Nicméně pro běžné účely je výhodnější použít jiný nástroj (viz již zmíněné nástroje Numba, mypyc, PyPy atd.).

3.3.5 Cython

Jedním z nejznámějších a pravděpodobně i nejpoužívanějších projektů, které se zaměřují na překlad Pythonu do nativního kódu, patří nástroj nazvaný *Cython* (pozor: neplést s podobně pojmenovaným *CPythonem*, což je, jak již dobře víme, označení klasického interpretu Pythonu). Tento překladač pracuje poněkud odlišným způsobem než výše zmíněný RPython. Zatímco se totiž RPython snaží odvozovat datové typy proměnných, parametrů i návratových hodnot funkcí na základě analýzy grafu (*CFG*), používá Cython přímý překlad Pythonu do programovacího jazyka C.

Ve chvíli, kdy Cython z nějakého důvodu nemá k dispozici informaci o datovém typu funkce/proměnné/parametru, použije `PyObject *`, tedy ukazatel na datovou strukturu reprezentující v klasickém Pythonu libovolnou hodnotu. V dalším kódu je samozřejmě nutné z tohoto objektu získat skutečnou hodnotu. I takto vlastně velmi primitivně provedený překlad dokáže ve skutečnosti výsledný kód přeloženého projektu zrychlit, což je ostatně patrné i na výsledku benchmarků, o nichž se zmíníme dále.

Cython jde ale ještě dále, protože rozšiřuje syntaxi programovacího jazyka Python o další klíčová slova, především pak o klíčové slovo `cdef`. Toto klíčové slovo je možné použít pro přesnou specifikaci typu proměnné či parametru, a to způsobem, který plně vychází z programovacího jazyka C. Tato typová informace samozřejmě umožňuje provedení dalších optimalizací ve výsledném kódu (opět lze vidět na výsledcích benchmarků uvedených v dalších částech knihy).

3.4 Python ve webových prohlížečích

Programovací jazyk Python se často využívá i při tvorbě webových služeb a webových aplikací. V této oblasti se Python nasazuje především na *back endu*, tj. pro tu část webové služby popř. aplikace, která je provozována na serveru a nějakým způsobem komunikuje s ostatními komponentami vytvářeného systému: *front endem* (typicky webový prohlížeč s interpretrem JavaScriptu), databází a dalšími (mikro)službami, například s využitím message brokerů, systémů pro monitoring a správu událostí atd. Tato oblast informatiky samozřejmě není pokryta pouze Pythonem, ale najdeme zde i mnohé další programovací jazyky a technologie, zejména Javu (a celý její *middleware*), JavaScript či TypeScript (`node.js`) a dnes taktéž programovací jazyk Go.

Zatímco pozice Pythonu v oblasti back endu je poměrně zřejmá, je situace na front endu mnohem složitější. Důvod je jednoduchý – v této oblasti z historických důvodů kraluje JavaScript, přičemž veškeré snahy a náhrady tohoto programovacího jazyka byly prozatím neúspěšné (pokusů bylo

hned několik, připomeňme například dnes již pozapomenuté webové prohlížeče s podporou jazyků TCL, VBScript či projekt Dark. Ovšem stále se můžeme setkat se snahami JavaScript nepoužívat přímo, popř. ho obejít s využitím jiných nástrojů. Příkladem mohou být technologie umožňující tvorbu webových aplikací podobným způsobem, jakoby se jednalo o aplikace desktopové.

3.4.1 Brython

Pro vývoj webových aplikací lze v některých případech využít nástroj nazvaný *Brython*. Jedná se o takzvaný transpřekladač (*transcompiler*, *transpiler*) z Pythonu do JavaScriptu, přičemž samotný překlad (resp. přesněji řečeno transpřeklad) probíhá na pozadí. To v praxi znamená, že přímo v kódu webové stránky je umístěn (či načítán) kód v Pythonu. Výhodou Brythonu je, že se jakékoli úpravy ihned projeví v aplikaci po znovunačtení stránky (stisk klávesy F5 ve webovém prohlížeči). Není tedy vyžadován explicitní krok s voláním (trans)překladače.

3.4.2 Transcrypt

Transcrypt je taktéž transpřekladačem Pythonu (konkrétně Pythonu 3) do JavaScriptu. Samotný transpřeklad je napsaný velmi kvalitně, takže lze zhruba odhadnout, že velikost vygenerovaného kódu v JavaScriptu odpovídá velikosti původního kódu napsaného v Pythonu (případně zvětšení je o jednotky procent). Musíme však počítat s tím, že je nutné načíst i jádro *Transcryptu*, jehož velikost je přibližně 20 kB. Tedy čím větší je kód samotné aplikace, tím (poměrově) menší budou režijní náklady. Zajímavá a užitečná je i podpora *type hintů* (informací o datovém typu parametrů, proměnných).

3.4.3 PyScript

PyScript je realizován tak, že se zdrojové kódy standardního CPythonu přeloží do WebAssembly (*WASM*), jenž je doplněn podpůrnými skripty naprogramovanými v JavaScriptu. Výsledkem je Python spustitelný ve webovém prohlížeči. Podobným způsobem lze namísto CPythonu použít i *MicroPython*.

Poznámka: s projekty Brython a PyScript se ještě setkáme v deváté kapitole.

3.5 MicroPython

MicroPython představuje alternativní implementaci Pythonu naprogramovanou taktéž v programovacím jazyku C, podobně jako standardní CPython. Ovšem již název tohoto projektu napovídá, že se jedná o variantu Pythonu určenou pro běh na šestnáctibitových a 32bitových mikrořa-

dičích, které mají omezený jak výpočetní výkon, tak i kapacitu ROM a RAM (v tomto kontextu se bavíme například o desítkách či stovkách *kilobajtů* a nikoli o jednotkách a desítkách *gigabajtů*). Mj. i z tohoto důvodu je syntaxe a sémantika omezena tak, že je MicroPython sice kompatibilní s Pythonem 3.4 (a částečně i Pythonem 3.5), ovšem většinu nových vlastností, které budou popsány ve čtvrté kapitole, zde již nenajdeme.

4 Postupné rozšiřování syntaxe a sémantiky Pythonu

4.1 Formátovací řetězce

Do programovacího jazyka Python byla v rámci nových vlastností verze 3.6 byla přidána podpora pro takzvané *formátovací řetězce* neboli zkráceně *f-řetězce* (*f-strings*). Tato nová konstrukce zjednodušuje formátování dat, která se mají vytisknout na standardní výstup, zobrazit na HTML stránce, zobrazit v grafickém uživatelském rozhraní nějakého projektu s GUI atd. Tyto operace typicky znamenají že se musí nějakým způsobem zkombinovat statický textový obsah s hodnotami proměnných, atributy objektů atd.

Formátovací řetězce doplňují a rozšiřují metodu *format* definovanou pro datový typ řetězec (*str*). Připomeňme si, jak se vlastně tato metoda používá:

```
# Formátování výstupu s využitím metody str.format

a = 1
b = 2
c = a + b

# výpis obsahu řetězce po doplnění hodnot dle šablony
print("{}+{}={}".format(a, b, c))
```

Zdrojový kód tohoto příkladu můžete najít na adrese
https://github.com/tisnik/Evoluce_Pythonu_priklady/blob/master/f-strings/string-format.py

Formátovací řetězce se zapisují podobně jako běžné řetězce, tj. jsou zapsané do apostrofů nebo uvozovek, popř. do trojitých apostrofů nebo uvozovek (víceřádkové řetězce). Ovšem oproti běžným řetězcům začínají formátovací řetězce prefixem *f* zapsaným před samotným řetězcovým literálem (konstantou) a právě z tohoto důvodu se v původní anglické dokumentaci o programovacím jazyce Python tyto řetězce nazývají *f-strings*.

Poznámka: to, že do programovacího jazyka Python byla přidána podpora pro formátovací řetězce, ovšem v žádném případě neznamena, že už není povoleno použít původní způsoby formátování výstupů. Ty jsou pochopitelně stále dostupné a záleží pouze na vás, zda formátovací řetězce budeme chtít použít či nikoli. Můžeme pochopitelně použít i přetížený operátor %:

```
# Formátování výstupu s využitím přetíženého operátoru %
```

```
a = 1
b = 2
c = a + b
```

```
# výpis obsahu řetězce po doplnění hodnot dle šablony
print("%d+%d=%d" % (a, b, c))
```

Zdrojový kód tohoto příkladu můžete najít na adrese

[https://github.com/tisnik/Evoluce_Pythonu_prikлады/blob/master/f-strings/percent-operator.py](https://github.com/tisnik/Evoluce_Pythonu_prikklady/blob/master/f-strings/percent-operator.py)

Poznámka: v dalších kapitolách si představíme nástroj Mypy, který nás v případě správného nastavení upozorní na to, ve kterých místech programového kódu by bylo vhodné namísto starších metod formátování řetězců použít f-řetězce.

4.1.1 Základní použití formátovacích řetězců pro tzv. interpolaci proměnných

Podívejme se nyní na jeden z nejjednodušších způsobů využití formátovacích řetězců. V následujícím skriptu napsaném v Pythonu nejdříve nadeklaruje tři proměnné a posléze vypíše hodnoty všech třech proměnných, a to podle šablony (*template*), která vypadá následovně:

```
f"{a}+{b}={c}"
```

Znaky `a`, `b` a `c` (které představují názvy proměnných) jsou v šabloně zapsány mezi složené závorky a z tohoto důvodu jsou při konstrukci výsledného řetězce tyto znaky nahrazeny za hodnotu proměnných se stejným jménem:

```
# Formátovací řetězce v Pythonu:
# - formátování výstupu založené na f-řetězci
# - dosazení hodnot proměnných do výsledného řetězce
```

```
a = 1
b = 2
c = a + b
```

```
# výpis obsahu řetězce po doplnění hodnot dle šablony
print(f"{a}+{b}={c}")
```

Zdrojový kód tohoto příkladu můžete najít na adrese
[https://github.com/tisnik/Evoluce_Pythonu_prikлады/blob/master/f-strings/f-string-1.py](https://github.com/tisnik/Evoluce_Pythonu_prikklady/blob/master/f-strings/f-string-1.py)

Po spuštění tohoto skriptu se na terminálu zobrazí následující text:

```
1+2=3
```

4.1.2 Chování Pythonu v případě, že se neuvede prefix „f“

Povšimněte si, že v předchozím skriptu je před řetězovým literálem skutečně uveden znak `f` (takzvaný *prefix*). V případě, že tento prefix neuvedeme (například na to zapomeneme), nebude formátování (tedy výše zmíněná „interpolace“ proměnných) provedeno, protože řetězec bez prefixu `f` programovací jazyk Python považuje za běžný řetězcový literál. Ostatně si toto chování můžeme velmi snadno otestovat:

```
# Formátovací řetězce v Pythonu:  
# - chování Pythonu v případě, že se použije běžný řetězec  
#   a nikoli f-řetězec  
  
a = 1  
b = 2  
c = a + b  
  
# výpis obsahu řetězce po doplnění hodnot dle šablony  
print("{a}+{b}={c}")
```

Zdrojový kód tohoto příkladu můžete najít na adrese
[https://github.com/tisnik/Evoluce_Pythonu_prikлады/blob/master/f-strings/f-string-2.py](https://github.com/tisnik/Evoluce_Pythonu_prikklady/blob/master/f-strings/f-string-2.py)

Po spuštění takto upraveného skriptu se zobrazí původní řetězec:

```
{a}+{b}={c}
```

Poznámka: to tedy znamená, že přidáním f-řetězců do Pythonu se nenarušila kompatibilita se staršími zdrojovými kódy.

4.1.3 Výrazy ve formátovacím řetězci

Zajímavé a taktéž užitečné je, že ve formátovacím řetězci mohou být uvedeny nejenom názvy proměnných, ale i složitější výrazy. To například znamená, že výpočet součtu hodnot proměnných a a b nemusíme ukládat do nové pomocné proměnné c , ale namísto toho můžeme součet zapsat přímo do formátovacího řetězce. Upravený skript vypadá následovně:

```
# Formátovací řetězce v Pythonu:  
# - výraz použitý v f-řetězci  
  
a = 1  
b = 2  
  
# výpis obsahu řetězce po doplnění hodnot dle šablony  
print(f"{a}+{b}={a+b}")
```

Zdrojový kód tohoto příkladu můžete najít na adrese
https://github.com/tisnik/Evoluce_Pythonu_priklady/blob/master/f-strings/f-string-3.py

Výsledek vytištěný na terminál po spuštění tohoto skriptu:

```
1+2=3
```

4.1.4 Podmínka ve výrazu zapsaném ve formátovacím řetězci

V programovacím jazyku Python je umožněno do výrazu zapsat i podmínku (v některých jiných jazycích je pro tento účel použit takzvaný *ternární operátor* popř. tuto možnost vůbec nemají). Z tohoto důvodu lze podmínku zapsat i do formátovacího řetězce, i když je vhodné upozornit na to, že výsledek už nemusí být příliš čitelný. Na druhou stranu lze tímto způsobem realizovat jednoduchý šablonovací systém:

```
# Formátovací řetězce v Pythonu:  
# - výraz s podmínkou použitý v f-řetězci  
  
a = 1  
b = -1  
  
# výpis obsahu řetězce po doplnění hodnot dle šablony  
print(f"Kladné: {'ano' if a>0 else 'ne'}")  
  
# výpis obsahu řetězce po doplnění hodnot dle šablony  
print(f"Kladné: {'ano' if b>0 else 'ne'}")
```

Zdrojový kód tohoto příkladu můžete najít na adrese
https://github.com/tisnik/Evoluce_Pythonu_priklady/blob/master/f-strings/f-string-1.py

Po spuštění tohoto skriptu se na terminálu zobrazí následující text:

```
1+2=3
```

4.1.2 Chování Pythonu v případě, že se neuvede prefix „f“

Povšimněte si, že v předchozím skriptu je před řetězovým literálem skutečně uveden znak `f` (takzvaný *prefix*). V případě, že tento prefix neuvedeme (například na to zapomeneme), nebude formátování (tedy výše zmíněná „interpolace“ proměnných) provedeno, protože řetězec bez prefixu `f` programovací jazyk Python považuje za běžný řetězcový literál. Ostatně si toto chování můžeme velmi snadno otestovat:

```
# Formátovací řetězce v Pythonu:  
# - chování Pythonu v případě, že se použije běžný řetězec  
#   a nikoli f-řetězec  
  
a = 1  
b = 2  
c = a + b  
  
# výpis obsahu řetězce po doplnění hodnot dle šablony  
print("{a}+{b}={c}")
```

Zdrojový kód tohoto příkladu můžete najít na adrese
https://github.com/tisnik/Evoluce_Pythonu_priklady/blob/master/f-strings/f-string-2.py

Po spuštění takto upraveného skriptu se zobrazí původní řetězec:

```
{a}+{b}={c}
```

Poznámka: to tedy znamená, že přidáním f-řetězců do Pythonu se nenarušila kompatibilita se staršími zdrojovými kódy.

4.1.3 Výrazy ve formátovacím řetězci

Zajímavé a taktéž užitečné je, že ve formátovacím řetězci mohou být uvedeny nejenom názvy proměnných, ale i složitější výrazy. To například znamená, že výpočet součtu hodnot proměnných a a b nemusíme ukládat do nové pomocné proměnné c , ale namísto toho můžeme součet zapsat přímo do formátovacího řetězce. Upravený skript vypadá následovně:

```
# Formátovací řetězce v Pythonu:
# - výraz použitý v f-řetězci

a = 1
b = 2

# výpis obsahu řetězce po doplnění hodnot dle šablony
print(f"{a}+{b}={a+b}")
```

Zdrojový kód tohoto příkladu můžete najít na adrese
https://github.com/tisnik/Evoluce_Pythonu_priklady/blob/master/f-strings/f-string-3.py

Výsledek vytištěný na terminál po spuštění tohoto skriptu:

```
1+2=3
```

4.1.4 Podmínka ve výrazu zapsaném ve formátovacím řetězci

V programovacím jazyku Python je umožněno do výrazu zapsat i podmínku (v některých jiných jazycích je pro tento účel použit takzvaný *ternární operátor* popř. tuto možnost vůbec nemají). Z tohoto důvodu lze podmínku zapsat i do formátovacího řetězce, i když je vhodné upozornit na to, že výsledek už nemusí být příliš čitelný. Na druhou stranu lze tímto způsobem realizovat jednoduchý šablonovací systém:

```
# Formátovací řetězce v Pythonu:
# - výraz s podmínkou použitý v f-řetězci

a = 1
b = -1

# výpis obsahu řetězce po doplnění hodnot dle šablony
print(f"Kladné: {'ano' if a>0 else 'ne'}")

# výpis obsahu řetězce po doplnění hodnot dle šablony
print(f"Kladné: {'ano' if b>0 else 'ne'}")
```

Zdrojový kód tohoto příkladu můžete najít na adrese
https://github.com/tisnik/Evoluce_Pythonu_priklady/blob/master/f-strings/f-string-4.py

Po spuštění tohoto skriptu by se na terminál měla vypsat následující dvojice zpráv:

Kladné: ano

Kladné: ne

4.1.5 Volání funkce ve formátovacím řetězci

Vzhledem k tomu, že se volání funkce může objevit v běžném výrazu, znamená to, že volání funkce lze realizovat i ve formátovacím řetězci. Opět ovšem platí, že výsledkem nemusí být příliš čitelný programový kód (a navíc, což může vadit více, nemusí být takový kód rozeznán a kontrolován integrovanými vývojovými prostředími):

```
# Formátovací řetězce v Pythonu:
# - volání funkce v f-řetězci

x = "Hello world!"

# výpis obsahu řetězce po doplnění hodnot dle šablony
print(f"Délka řetězce '{x}' je {len(x)} znaků")
```

Zdrojový kód tohoto příkladu můžete najít na adrese
https://github.com/tisnik/Evoluce_Pythonu_priklady/blob/master/f-strings/f-string-5.py

Po spuštění takto upraveného skriptu by se měla spočítat délka řetězce uloženého v proměnné *x* a následně by se tato délka měla použít při formátování zprávy zobrazené na terminálu:

```
Délka řetězce 'Hello world!' je 12 znaků
```

4.1.6 Volání metody v f-řetězci

Podobně jako u volání funkcí můžeme přímo v rámci formátovacího řetězce realizovat volání metody nějakého objektu. V následujícím skriptu je ukázán způsob zavolání metody nazvané *capitalize* pro instanci třídy řetězec (*str*). Tato metoda převede první znak ve větě na velké písmeno (verzálku):

```
# Formátovací řetězce v Pythonu:
# - volání metody v f-řetězci
```

```
x = "hello world!"

# výpis obsahu řetězce po doplnění hodnot dle šablony
print(f"Důležitá zpráva pro vás: '{x.capitalize()}'")
```

Zdrojový kód tohoto příkladu můžete najít na adrese
https://github.com/tisnik/Evoluce_Pythonu_priklady/blob/master/f-strings/f-string-6.py

Výsledná zpráva zobrazená na terminálu bude vypadat následovně:

```
Důležitá zpráva pro vás: 'Hello world!'
```

4.1.7 Jednodušší ladění

Pro ladění programů vyvíjených v Pythonu je možné využít různé prostředky (ostatně se s některými z nich seznámíme v dalším textu), ovšem mnohdy je zdaleka nejsnazší si nechat vypsát obsah zvolených sledovaných proměnných, ideálně i s jejich názvy. I pro tento úkol je možné využít formátovací řetězce, protože pokud v řetězci použijeme zápis {proměnná=}, vloží se do výsledného řetězce jak název proměnné, tak i její hodnota. Podívejme se opět na jednoduchý demonstrační příklad, v němž si necháme vypsát název i hodnotu proměnných se jmény name a surname:

```
# Formátovací řetězce v Pythonu:
# - výpis jména proměnné společně s její hodnotou

name = "Guido"
surname = "Rossum"

# výpis obsahu řetězce po doplnění hodnot dle šablony
print(f"{name=} {surname=}")
```

Zdrojový kód tohoto příkladu můžete najít na adrese
https://github.com/tisnik/Evoluce_Pythonu_priklady/blob/master/f-strings/f-string-7.py

Výsledná zpráva zobrazená na terminálu by měla vypadat takto:

```
name='Guido' surname='Rossum'
```

4.1.8 Zarovnání hodnot ve výsledném naformátovaném řetězci

Při formátování hodnot, které mají být například součástí tabulek, vyvstává další problém: jak jednotlivé hodnoty zarovnat. I pro tento úkol je možné využít formátovací řetězce, protože za

Zdrojový kód tohoto příkladu můžete najít na adrese
https://github.com/tisnik/Evoluce_Pythonu_priklady/blob/master/f-strings/f-string-4.py

Po spuštění tohoto skriptu by se na terminál měla vypsát následující dvojice zpráv:

Kladné: ano

Kladné: ne

4.1.5 Volání funkce ve formátovacím řetězci

Vzhledem k tomu, že se volání funkce může objevit v běžném výrazu, znamená to, že volání funkce lze realizovat i ve formátovacím řetězci. Opět ovšem platí, že výsledkem nemusí být příliš čitelný programový kód (a navíc, což může vadit více, nemusí být takový kód rozeznán a kontrolován integrovanými vývojovými prostředími):

```
# Formátovací řetězce v Pythonu:
# - volání funkce v f-řetězci

x = "Hello world!"

# výpis obsahu řetězce po doplnění hodnot dle šablony
print(f"Délka řetězce '{x}' je {len(x)} znaků")
```

Zdrojový kód tohoto příkladu můžete najít na adrese
https://github.com/tisnik/Evoluce_Pythonu_priklady/blob/master/f-strings/f-string-5.py

Po spuštění takto upraveného skriptu by se měla spočítat délka řetězce uloženého v proměnné *x* a následně by se tato délka měla použít při formátování zprávy zobrazené na terminálu:

Délka řetězce 'Hello world!' je 12 znaků

4.1.6 Volání metody v f-řetězci

Podobně jako u volání funkcí můžeme přímo v rámci formátovacího řetězce realizovat volání metody nějakého objektu. V následujícím skriptu je ukázán způsob zavolání metody nazvané *capitalize* pro instanci třídy řetězec (*str*). Tato metoda převede první znak ve větě na velké písmeno (verzálku):

```
# Formátovací řetězce v Pythonu:
# - volání metody v f-řetězci
```

```
x = "hello world!"

# výpis obsahu řetězce po doplnění hodnot dle šablony
print(f"Důležitá zpráva pro vás: '{x.capitalize()}'")
```

Zdrojový kód tohoto příkladu můžete najít na adrese
https://github.com/tisnik/Evoluce_Pythonu_priklady/blob/master/f-strings/f-string-6.py

Výsledná zpráva zobrazena na terminálu bude vypadat následovně:

```
Důležitá zpráva pro vás: 'Hello world!'
```

4.1.7 Jednodušší ladění

Pro ladění programů vyvíjených v Pythonu je možné využít různé prostředky (ostatně se s některými z nich seznámíme v dalším textu), ovšem mnohdy je zdaleka nejsnazší si nechat vypsát obsah zvolených sledovaných proměnných, ideálně i s jejich názvy. I pro tento úkol je možné využít formátovací řetězce, protože pokud v řetězci použijeme zápis {proměnná=}, vloží se do výsledného řetězce jak název proměnné, tak i její hodnota. Podívejme se opět na jednoduchý demonstrační příklad, v němž si necháme vypsát název i hodnotu proměnných se jmény name a surname:

```
# Formátovací řetězce v Pythonu:
# - výpis jména proměnné společně s její hodnotou

name = "Guido"
surname = "Rossum"

# výpis obsahu řetězce po doplnění hodnot dle šablony
print(f"{name=} {surname=}")
```

Zdrojový kód tohoto příkladu můžete najít na adrese
https://github.com/tisnik/Evoluce_Pythonu_priklady/blob/master/f-strings/f-string-7.py

Výsledná zpráva zobrazená na terminálu by měla vypadat takto:

```
name='Guido' surname='Rossum'
```

4.1.8 Zarovnání hodnot ve výsledném naformátovaném řetězci

Při formátování hodnot, které mají být například součástí tabulek, vyvstává další problém: jak jednotlivé hodnoty zarovnat. I pro tento úkol je možné využít formátovací řetězce, protože za

jméno proměnné či za výraz lze v případě potřeby zapsat jak jeho požadovanou délku ve výsledku (například deset znaků), ale i to, zda má být daná hodnota zarovnána doleva, doprava či na střed. Pro tento účel lze ve formátovacím řetězci použít následující zápis:

```
f"{výraz : zarovnání šířka}"
```

Pro specifikaci *zarovnání* se používají znaky < pro zarovnání doleva, ^ pro vycentrování a > pro zarovnání doprava. Navíc je možné pro numerické hodnoty použít i znak =, který zajistí přidání výplně mezi znaménko a (kratší) numerickou hodnotu (podobně je tomu například v tabulkových procesorech). A specifikace *šířky* určuje počet znaků rezervovaných ve výsledném řetězci pro výpis hodnoty výrazu. Pokud je hodnota výrazu kratší, než zadaná šířka, je výraz zarovnán a doplněn mezerami (popř. u čísel typicky nulami).

Podívejme se na příklad, v němž budeme realizovat zobrazení tabulky obsahující tři sloupce. V této tabulce bude vypsán počet dětí, které se narodily s uvedeným jménem (a vyjadřuje tedy oblíbenost jmen). Hodnoty v prvním sloupci jsou vycentrovány (tedy zarovnány na střed), hodnoty ve druhém sloupci jsou zarovnány doprava a konečně hodnoty ve sloupci třetím jsou zarovnány doleva:

```
# Formátovací řetězce v Pythonu:
# - specifikace šířky hodnoty
# - specifikace zarovnání hodnoty
# - výpis tabulky s popularitou jmen

jmena = ("Eliška", "Tereza", "Anna", "Adéla", "Natálie", "Hana")
narození = (2332, 1900, 1708, 1535, 1386, 287)

# výpis obsahu řetězce po doplnění hodnot dle šablony
print(f"{'Pořadí' : ^10}{'Jméno' : <10}{'Narození' : >10}")

for i, (jmeno, nar) in enumerate(zip(jmena, narození)):
    # výpis obsahu řetězce po doplnění hodnot dle šablony
    print(f"{i : ^10}{jmeno : <10}{nar : >10}")
```

Zdrojový kód tohoto příkladu můžete najít na adrese
https://github.com/tisnik/Evoluce_Pythonu_priklady/blob/master/f-strings/f-string-8.py

A takto vypadá tabulka po svém zobrazení na terminálu:

Pořadí	Jméno	Narození
0	Eliška	2332
1	Tereza	1900
2	Anna	1708
3	Adéla	1535
4	Natálie	1386
5	Hana	287

4.1.9 Naformátování numerických hodnot

Velmi často se v praxi setkáme s nutností výpisu naformátovaných numerických hodnot, ať již se jedná o celá čísla nebo o hodnoty s plovoucí řádovou čárkou. Příkladem může být zobrazení tabulky s převrácenými hodnotami čísel 1 až 10. Můžeme se pokusit zobrazit hodnoty zarovnané doleva, přičemž šířka každého sloupce bude nastavena na pět znaků:

```
# Formátovací řetězce v Pythonu:
# - výpis tabulky převrácených hodnot - základní varianta
# - zarovnání hodnot
# - specifikace šířky hodnot

# výpis obsahu řetězce po doplnění hodnot dle šablony
print(f'{x' : >5}{1/x' : >5}")

for x in range(1, 11):
    y = 1 / x
    # výpis obsahu řetězce po doplnění hodnot dle šablony
    print(f"{x : >5} {y : >5}")
```

Zdrojový kód tohoto příkladu můžete najít na adrese

https://github.com/tisnik/Evoluce_Pythonu_priklady/blob/master/f-strings/f-string-9.py

Výsledkem bude tato tabulka:

```
x  1/x
1   1.0
2   0.5
3  0.3333333333333333
4   0.25
5   0.2
6  0.1666666666666666
7  0.14285714285714285
8   0.125
9  0.1111111111111111
10  0.1
```

jméno proměnné či za výraz lze v případě potřeby zapsat jak jeho požadovanou délku ve výsledku (například deset znaků), ale i to, zda má být daná hodnota zarovnána doleva, doprava či na střed. Pro tento účel lze ve formátovacím řetězci použít následující zápis:

```
f"{výraz : zarovnání šířka}"
```

Pro specifikaci *zarovnání* se používají znaky < pro zarovnání doleva, ^ pro vycentrování a > pro zarovnání doprava. Navíc je možné pro numerické hodnoty použít i znak =, který zajistí přidání výplně mezi znaménko a (kratší) numerickou hodnotu (podobně je tomu například v tabulkových procesorech). A specifikace *šířky* určuje počet znaků rezervovaných ve výsledném řetězci pro výpis hodnoty výrazu. Pokud je hodnota výrazu kratší, než zadaná šířka, je výraz zarovnán a doplněn mezerami (popř. u čísel typicky nulami).

Podívejme se na příklad, v němž budeme realizovat zobrazení tabulky obsahující tři sloupce. V této tabulce bude vypsán počet dětí, které se narodily s uvedeným jménem (a vyjadřuje tedy oblíbenost jmen). Hodnoty v prvním sloupci jsou vycentrovány (tedy zarovnány na střed), hodnoty ve druhém sloupci jsou zarovnány doprava a konečně hodnoty ve sloupci třetím jsou zarovnány doleva:

```
# Formátovací řetězce v Pythonu:
# - specifikace šířky hodnoty
# - specifikace zarovnání hodnoty
# - výpis tabulky s popularitou jmen

jmena = ("Eliška", "Tereza", "Anna", "Adéla", "Natálie", "Hana")
narozeni = (2332, 1900, 1708, 1535, 1386, 287)

# výpis obsahu řetězce po doplnění hodnot dle šablony
print(f"{'Pořadí' : ^10}{'Jméno' : <10}{'Narození' : >10}")

for i, (jmeno, nar) in enumerate(zip(jmena, narozeni)):
    # výpis obsahu řetězce po doplnění hodnot dle šablony
    print(f"{i : ^10}{jmeno : <10}{nar : >10}")
```

Zdrojový kód tohoto příkladu můžete najít na adrese
https://github.com/tisnik/Evoluce_Pythonu_priklady/blob/master/f-strings/f-string-8.py

A takto vypadá tabulka po svém zobrazení na terminálu:

Pořadí	Jméno	Narození
0	Eliška	2332
1	Tereza	1900
2	Anna	1708
3	Adéla	1535
4	Natálie	1386
5	Hana	287

4.1.9 Naformátování numerických hodnot

Velmi často se v praxi setkáme s nutností výpisu naformátovaných numerických hodnot, ať již se jedná o celá čísla nebo o hodnoty s plovoucí řádovou čárkou. Příkladem může být zobrazení tabulky s převrácenými hodnotami čísel 1 až 10. Můžeme se pokusit zobrazit hodnoty zarovnané doleva, přičemž šířka každého sloupce bude nastavena na pět znaků:

```
# Formátovací řetězce v Pythonu:
# - výpis tabulky převrácených hodnot - základní varianta
# - zarovnání hodnot
# - specifikace šířky hodnot

# výpis obsahu řetězce po doplnění hodnot dle šablony
print(f"{'x' : >5}{'1/x' : >5}")

for x in range(1, 11):
    y = 1 / x
    # výpis obsahu řetězce po doplnění hodnot dle šablony
    print(f"{'x' : >5} {'y' : >5}")
```

Zdrojový kód tohoto příkladu můžete najít na adrese

[https://github.com/tisnik/Evoluce_Pythonu_prikлады/blob/master/f-strings/f-string-9.py](https://github.com/tisnik/Evoluce_Pythonu_prikklady/blob/master/f-strings/f-string-9.py)

Výsledkem bude tato tabulka:

```
x  1/x
1   1.0
2   0.5
3  0.3333333333333333
4   0.25
5   0.2
6  0.1666666666666666
7  0.14285714285714285
8  0.125
9  0.1111111111111111
10  0.1
```

Jak je však patrné, není tato tabulka ani příliš pěkná ani přehledná. Je to mj. způsobeno tím, že některé výsledky (1/3.0 atd.) mají (alespoň teoreticky) nekonečný počet cifer a subsystém pro naformátování řetězce se je pokusí zobrazit s maximálním možným počtem cifer. Ten ovšem převyšuje šířku sloupce (pět znaků) a proto nejsou hodnoty zarovnány a tabulka je špatně čitelná:

Specifikace šířky i počtu cifer za desetinnou tečkou

Ve skutečnosti je však možné ve formátovacím parametru *šířka* specifikovat nejenom šířku sloupce, ale i maximální počet cifer zobrazených za desetinnou tečkou. To znamená, že můžeme specifikovat, že celková šířka sloupce má být rovna pěti znakům a za desetinnou tečkou mají být zobrazeny maximálně dva znaky, nezávisle na konkrétní zobrazované hodnotě:

```
# Formátovací řetězce v Pythonu:
# - výpis tabulky převrácených hodnot
# - varianta se šířkami sloupců
#   a maximálním počtem cifer za desetinnou tečkou

# výpis obsahu řetězce po doplnění hodnot dle šablony
print(f"{'x' : >5}{'1/x' : >5}")

for x in range(1, 11):
    y = 1 / x
    # výpis obsahu řetězce po doplnění hodnot dle šablony
    print(f"{'x' : 5} {'y' : 5.2}")
```

Zdrojový kód tohoto příkladu můžete najít na adrese
https://github.com/tisnik/Evoluce_Pythonu_priklady/blob/master/f-strings/f-string-A.py

Výsledek, který získáme po spuštění tohoto skriptu, je sice již lepší, ale stále není ideální, protože subsystém pro formátování řetězců u výsledků s menším počtem cifer za desetinnou tečkou nedoplnil nuly pro správné zarovnání:

```
x  1/x
1   1.0
2   0.5
3  0.33
4  0.25
5   0.2
6  0.17
7  0.14
8  0.12
9  0.11
10  0.1
```

Specifikace typu reprezentace hodnoty

Vypomoci si můžeme doplněním specifikace formátu o *typ* (re)prezentace hodnoty. V našem konkrétním případě doplníme typ *f*, což znamená *fixed point* (zobrazení hodnoty s pevným počtem cifer za desetinnou tečkou):

```
f"{výraz : zarovnání šířka typ}"
```

V praxi může použití vypadat takto:

```
# Formátovací řetězce v Pythonu:
# - výpis tabulky převrácených hodnot
# - varianta se šířkami sloupců
#   a přesným počtem cifer za desetinnou tečkou

# výpis obsahu řetězce po doplnění hodnot dle šablony
print(f'{'x' : >5}{'1/x' : >5}')
```

```
for x in range(1, 11):
    y = 1 / x
    # výpis obsahu řetězce po doplnění hodnot dle šablony
    print(f'{'x : 5} {'y : 5.2f}')
```

Zdrojový kód tohoto příkladu můžete najít na adrese

https://github.com/tisnik/Evoluce_Pythonu_priklady/blob/master/f-strings/f-string-B.py

Výsledná tabulka je v tomto případě již velmi dobře čitelná, i když pochopitelně některé hodnoty jsou hrubě nepřesné (1/3.0, 1/7.0 atd.):

```
x  1/x
1  1.00
2  0.50
3  0.33
4  0.25
5  0.20
6  0.17
7  0.14
8  0.12
9  0.11
10 0.10
```


Jak je však patrné, není tato tabulka ani příliš pěkná ani přehledná. Je to mj. způsobeno tím, že některé výsledky (1/3.0 atd.) mají (alespoň teoreticky) nekonečný počet cifer a subsystém pro naformátování řetězce se je pokusí zobrazit s maximálním možným počtem cifer. Ten ovšem převyšuje šířku sloupce (pět znaků) a proto nejsou hodnoty zarovnány a tabulka je špatně čitelná:

Specifikace šířky i počtu cifer za desetinnou tečkou

Ve skutečnosti je však možné ve formátovacím parametru *šířka* specifikovat nejenom šířku sloupce, ale i maximální počet cifer zobrazených za desetinnou tečkou. To znamená, že můžeme specifikovat, že celková šířka sloupce má být rovna pěti znakům a za desetinnou tečkou mají být zobrazeny maximálně dva znaky, nezávisle na konkrétní zobrazované hodnotě:

```
# Formátovací řetězce v Pythonu:
# - výpis tabulky převrácených hodnot
# - varianta se šířkami sloupců
#   a maximálním počtem cifer za desetinnou tečkou

# výpis obsahu řetězce po doplnění hodnot dle šablony
print(f"{'x' : >5}{'1/x' : >5}")

for x in range(1, 11):
    y = 1 / x
    # výpis obsahu řetězce po doplnění hodnot dle šablony
    print(f"{'x' : 5} {'y' : 5.2}")
```

Zdrojový kód tohoto příkladu můžete najít na adrese
https://github.com/tisnik/Evoluce_Pythonu_priklady/blob/master/f-strings/f-string-A.py

Výsledek, který získáme po spuštění tohoto skriptu, je sice již lepší, ale stále není ideální, protože subsystém pro formátování řetězců u výsledků s menším počtem cifer za desetinnou tečkou nedoplnil nuly pro správné zarovnání:

x	1/x
1	1.0
2	0.5
3	0.33
4	0.25
5	0.2
6	0.17
7	0.14
8	0.12
9	0.11
10	0.1

Specifikace typu reprezentace hodnoty

Vypomoci si můžeme doplněním specifikace formátu o *typ* (re)prezentace hodnoty. V našem konkrétním případě doplníme typ *f*, což znamená *fixed point* (zobrazení hodnoty s pevným počtem cifer za desetinnou tečkou):

```
f"{výraz : zarovnání šířka typ}"
```

V praxi může použití vypadat takto:

```
# Formátovací řetězce v Pythonu:
# - výpis tabulky převrácených hodnot
# - varianta se šířkami sloupců
#   a přesným počtem cifer za desetinnou tečkou

# výpis obsahu řetězce po doplnění hodnot dle šablony
print(f"{'x' : >5}{'1/x' : >5}")

for x in range(1, 11):
    y = 1 / x
    # výpis obsahu řetězce po doplnění hodnot dle šablony
    print(f"{x : 5} {y : 5.2f}")
```

Zdrojový kód tohoto příkladu můžete najít na adrese

https://github.com/tisnik/Evoluce_Pythonu_priklady/blob/master/f-strings/f-string-B.py

Výsledná tabulka je v tomto případě již velmi dobře čitelná, i když pochopitelně některé hodnoty jsou hrubě nepřesné (1/3.0, 1/7.0 atd.):

x	1/x
1	1.00
2	0.50
3	0.33
4	0.25
5	0.20
6	0.17
7	0.14
8	0.12
9	0.11
10	0.10

Zobrazení hodnoty v procentech

Mnohdy je též nutné zobrazit hodnotu uvedenou v procentech, což vlastně znamená hodnotu vynásobenou stem. Tento výpočet není nutné zapisovat do programu explicitně, neboť ve formátovacím řetězci můžeme použít typ %. Tento typ zajistí, že je hodnota nejprve interně vynásobena stem a posléze zobrazena.

Ukažme si to na tabulce s výpočtem složeného úroku. Ve druhém sloupci tabulky je vypsán úrok (tj. hodnota mezi 0,1 a 1,0), který je ovšem zobrazen formou procent (tedy namísto 0,5 se vypíše 50% atd.):

```
# Formátovací řetězce v Pythonu:
# - výpis tabulky, jejíž jeden sloupec obsahuje hodnoty
#   vypsané formou procent

# výpis obsahu řetězce po doplnění hodnot dle šablony
print(f"{'částka' : >5} {'úrok' : >5} {'za rok' : >7} {'za 5 let' : >7}")

castka = 1000

for i in range(1, 11):
    urok = i / 10.0
    za_rok = castka * (1 + urok)
    za_5_let = castka * (1 + urok) ** 5
    # výpis obsahu řetězce po doplnění hodnot dle šablony
    print(f"{castka : >5} {urok : 5.0%} {za_rok : >7.0f} {za_5_let : > 7.0f}")
```

Zdrojový kód tohoto příkladu můžete najít na adrese
https://github.com/tisnik/Evoluce_Pythonu_priklady/blob/master/f-strings/f-string-C.py

částka	úrok	za rok	za 5 let
1000	10%	1100	1611
1000	20%	1200	2488
1000	30%	1300	3713
1000	40%	1400	5378
1000	50%	1500	7594
1000	60%	1600	10486
1000	70%	1700	14199
1000	80%	1800	18896
1000	90%	1900	24761
1000	100%	2000	32000

4.1.10 Zobrazení znaků a celočíselných hodnot ve zvolené číselné soustavě

V posledním demonstračním příkladu, v němž si ukážeme formátování řetězců, si zobrazíme tabulku s hodnotami 64 až 95. Tyto hodnoty budou zobrazeny dekadicky, v dvojkové soustavě, v osmičkové soustavě, v hexadecimální soustavě (a to dokonce ve dvou variantách, které se liší velikostí znaků 'a' až 'f') a konečně se celočíselná hodnota převede na ASCII znak:

```
# Formátovací řetězce v Pythonu:
# - výpis celočíselných hodnot 64 až 95 decimálně, binárně,
#   v osmičkové soustavě, v hexadecimální soustavě a taktéž
#   formou ASCII znaku

# výpis obsahu řetězce po doplnění hodnot dle šablony
print(f"{'DEC'} {'BIN' : ^7} {'OCT' : >3} {'hex' : >2} {'HEX' : >2} {'ASCII'}")

for i in range(64, 64 + 32):
    # výpis obsahu řetězce po doplnění hodnot dle šablony
    print(f"{i : >2d} {i : 07b} {i : >3o} {i : >2x} {i : >2X} {i : >c}")
```

Zdrojový kód tohoto příkladu můžete najít na adrese

https://github.com/tisnik/Evoluce_Pythonu_priklady/blob/master/f-strings/f-string-D.py

Tabulka, která se zobrazí po spuštění výše uvedeného skriptu, by měla vypadat takto:

DEC	BIN	OCT	hex	HEX	ASCII
64	1000000	100	40	40	@
65	1000001	101	41	41	A
66	1000010	102	42	42	B
67	1000011	103	43	43	C
68	1000100	104	44	44	D
69	1000101	105	45	45	E
70	1000110	106	46	46	F
71	1000111	107	47	47	G
72	1001000	110	48	48	H
73	1001001	111	49	49	I
74	1001010	112	4a	4A	J
75	1001011	113	4b	4B	K
76	1001100	114	4c	4C	L
77	1001101	115	4d	4D	M
78	1001110	116	4e	4E	N
79	1001111	117	4f	4F	O

Zobrazení hodnoty v procentech

Mnohdy je též nutné zobrazit hodnotu uvedenou v procentech, což vlastně znamená hodnotu vynásobenou stem. Tento výpočet není nutné zapisovat do programu explicitně, neboť ve formátovacím řetězci můžeme použít typ %. Tento typ zajistí, že je hodnota nejprve interně vynásobena stem a posléze zobrazena.

Ukažme si to na tabulce s výpočtem složeného úroku. Ve druhém sloupci tabulky je vypsán úrok (tj. hodnota mezi 0,1 a 1,0), který je ovšem zobrazen formou procent (tedy namísto 0,5 se vypíše 50% atd.):

```
# Formátovací řetězce v Pythonu:
# - výpis tabulky, jejíž jeden sloupec obsahuje hodnoty
#   vypsané formou procent

# výpis obsahu řetězce po doplnění hodnot dle šablony
print(f"{'částka' : >5} {'úrok' : >5} {'za rok' : >7} {'za 5 let' : >7}")

castka = 1000

for i in range(1, 11):
    urok = i / 10.0
    za_rok = castka * (1 + urok)
    za_5_let = castka * (1 + urok) ** 5
    # výpis obsahu řetězce po doplnění hodnot dle šablony
    print(f"{castka : >5} {urok : 5.0%} {za_rok : >7.0f} {za_5_let : > 7.0f}")
```

Zdrojový kód tohoto příkladu můžete najít na adrese
https://github.com/tisnik/Evoluce_Pythonu_priklady/blob/master/f-strings/f-string-C.py

částka	úrok	za rok	za 5 let
1000	10%	1100	1611
1000	20%	1200	2488
1000	30%	1300	3713
1000	40%	1400	5378
1000	50%	1500	7594
1000	60%	1600	10486
1000	70%	1700	14199
1000	80%	1800	18896
1000	90%	1900	24761
1000	100%	2000	32000

4.1.10 Zobrazení znaků a celočíselných hodnot ve zvolené číselné soustavě

V posledním demonstračním příkladu, v němž si ukážeme formátování řetězců, si zobrazíme tabulku s hodnotami 64 až 95. Tyto hodnoty budou zobrazeny dekadicky, v dvojkové soustavě, v osmičkové soustavě, v hexadecimální soustavě (a to dokonce ve dvou variantách, které se liší velikostí znaků 'a' až 'f') a konečně se celočíselná hodnota převede na ASCII znak:

```
# Formátovací řetězce v Pythonu:
# - výpis celočíselných hodnot 64 až 95 decimálně, binárně,
#   v osmičkové soustavě, v hexadecimální soustavě a taktéž
#   formou ASCII znaku

# výpis obsahu řetězce po doplnění hodnot dle šablony
print(f"{'DEC'} {'BIN' : ^7} {'OCT' : >3} {'hex' : >2} {'HEX' : >2} {'ASCII'}")

for i in range(64, 64 + 32):
    # výpis obsahu řetězce po doplnění hodnot dle šablony
    print(f"{i : >2d} {i : 07b} {i : >3o} {i : >2x} {i : >2X} {i : >c}")
```

Zdrojový kód tohoto příkladu můžete najít na adrese

https://github.com/tisnik/Evoluce_Pythonu_priklady/blob/master/f-strings/f-string-D.py

Tabulka, která se zobrazí po spuštění výše uvedeného skriptu, by měla vypadat takto:

DEC	BIN	OCT	hex	HEX	ASCII
64	1000000	100	40	40	@
65	1000001	101	41	41	A
66	1000010	102	42	42	B
67	1000011	103	43	43	C
68	1000100	104	44	44	D
69	1000101	105	45	45	E
70	1000110	106	46	46	F
71	1000111	107	47	47	G
72	1001000	110	48	48	H
73	1001001	111	49	49	I
74	1001010	112	4a	4A	J
75	1001011	113	4b	4B	K
76	1001100	114	4c	4C	L
77	1001101	115	4d	4D	M
78	1001110	116	4e	4E	N
79	1001111	117	4f	4F	O

80	1010000	120	50	50	P
81	1010001	121	51	51	Q
82	1010010	122	52	52	R
83	1010011	123	53	53	S
84	1010100	124	54	54	T
85	1010101	125	55	55	U
86	1010110	126	56	56	V
87	1010111	127	57	57	W
88	1011000	130	58	58	X
89	1011001	131	59	59	Y
90	1011010	132	5a	5A	Z
91	1011011	133	5b	5B	[
92	1011100	134	5c	5C	\
93	1011101	135	5d	5D	]
94	1011110	136	5e	5E	^
95	1011111	137	5f	5F	_

4.2 Klíčová slova `async` a `await`

V některých situacích je důležité nebo alespoň užitečné dosáhnout zvýšení efektivity vyvíjené aplikace (například zvýšit počet odpovědí, na které HTTP server stihne odpovědět za určitou časovou jednotku) a přitom není možné či vhodné využívat přímočaré, ale na druhou stranu dosti nízkoúrovňové řešení založené na použití většího množství vláken (*threads*) spravovaných systémem, nehledě na to, že se v tomto případě může negativně projevit existence GILu (*Global Interpreter Lock*).

Řešení těchto problémů může spočívat v tom, že nějakou vhodnou technologií umožníme asynchronní vykonávání kódu v těch okamžicích, v nichž jsou jiné části kódu pozastaveny, protože čekají na dokončení nějakých vstupně-výstupních operací. Naprosto typickým příkladem jsou některé virtuální stroje JavaScriptu, které povětšinou umožňují běh aplikace pouze v jediném vláknu, ale i v takových případech je *pro některé typy operací* umožněno zajistit souběh několika operací.

4.2.1 Korutiny

V případě, že v programovacím jazyku Python vyvíjíme aplikaci, která intenzivně používá vstupně-výstupní operace (*I/O operace*), tedy například zajišťuje přístup k datům přes protokol HTTP(s), provádí volání databázových operací, export či import dat ze souborů atd., lze namísto většího množství vláken využít takzvané *korutiny*, což jsou původně běžné funkce transformované takovým způsobem, aby mohly být vykonávány *souběžně*, nikoli však nutně *paralelně*.

Ve světě IT existuje poměrně velké množství způsobů konstrukce a volání korutin. V programovacím jazyku Python se (vedle již zavedeného klíčového slova *yield*) nově používají dvě klíčová slova *async* a *await*, přičemž *async* je klíčové slovo umožňující (resp. zajišťující) transformaci běžné funkce do korutiny a *await* je klíčové slovo určené pro realizaci čekání na dokončení korutiny se získáním její návratové hodnoty. Tato slova byla přidána do Pythonu verze 3.5 (ovšem tak, aby se zajistila zpětná kompatibilita – nejednalo se tedy prozatím o skutečná klíčová slova) a v Pythonu verze 3.7 se z nich oficiálně stala rezervovaná klíčová slova. V současnosti se tedy již jedná o známé a relativně často používané programové konstrukce.

Poznámka: v této kapitole si ukážeme pouze základy použití `async` a `await` společně se standardní knihovnou `asyncio`. Jedná se však o poměrně důležité a rozsáhlé téma, kterému bude věnována samostatná kapitola, protože programátoři využívající Python mají k dispozici relativně velké množství dalších podpůrných knihoven. Ty jsou však stále založeny na využití klíčových slov `async` a `await`.

4.2.2 Synchronní a asynchronní provádění úloh: analogie

Klíčová slova `async` a `await` byla do Pythonu (ale nejenom tam, nalezneme je totiž například i v JavaScriptu či Rustu) přidána z toho důvodu, aby umožnila definovat a spouštět některé úlohy asynchronně. Jedná se přitom o koncept, který může být zpočátku matoucí, protože počítače typicky spouští úlohy synchronně, tedy stylem: vykonej úlohu A, *potom* vykonej úlohu B a *následně* vykonej úlohu C. Rozdíl mezi synchronním a asynchronním zpracováním byl popsán v analogii dostupné na adrese <https://www.youtube.com/watch?v=iG6fr81xHKA&t=269s>, kterou si na tomto místě dovolím parafrázovat:

Předpokládejme, že nějaký šachista má na exhibici hrát postupné nebo simultánní partie (simultánku) proti 24 dalším hráčům. Přitom je v pravidlech soutěže stanoveno, že každý tah musí být tímto šachistou proveden do pěti sekund, zatímco jeho protihráči mají na každý tah 55 sekund. Průměrná hra přitom trvá 60 tahů (30 tahů každého hráče).

Pokud by se celý turnaj odehrával „synchronně“, odehrál by onen šachista hru s prvním protihráčem až do konce, poté s druhým hráčem a tak dále. Přitom každá hra trvá průměrně $(55 + 5) \cdot 30 = 1800$ sekund, tedy 30 minut. To ovšem znamená, že celá exhibice bude trvat velmi dlouho: $24 \cdot 30 = 720$ minut, neboli celých 12 hodin.

Asynchronní verze turnaje je založena na simultanci. Šachista se v tomto případě přesouvá od jedné šachovnice ke druhé, přičemž u každé šachovnice má jen pět sekund na provedení svého tahu. To znamená, že provedení jednoho tahu postupně na všech 24 šachovnicích mu bude trvat pouze $24 \cdot 5 = 120$ sekund neboli 2 minuty. A celá exhibice bude v takovém případě dohrána velmi rychle: $120 \cdot 30 = 3600$ sekund, což znamená pouze za jednu hodinu (oproti původním dvanácti